EQUIPO 4

CLOUDFLARE

Un proxy delante de nuestra API

Fundamentos de Arquitectura de Software y Desarrollo Backend

26 DE OCTUBRE DE 2026

UNIDAD II · EXPOSICIÓN

CONTENIDO

01 El problema

02 Qué resuelve

03 Las piezas

04 Cómo se usa

05 Con lo que ya vimos

06 Ejemplo con nuestra API

07 Ganancias y límites

08 Cuándo no · Fuentes

EQUIPO 4

JP

JZ

SH

DR

01 Josue Enmanuel Poot Mateo

Problema y qué resuelve

02 Juan Jose Zapata Buenfil

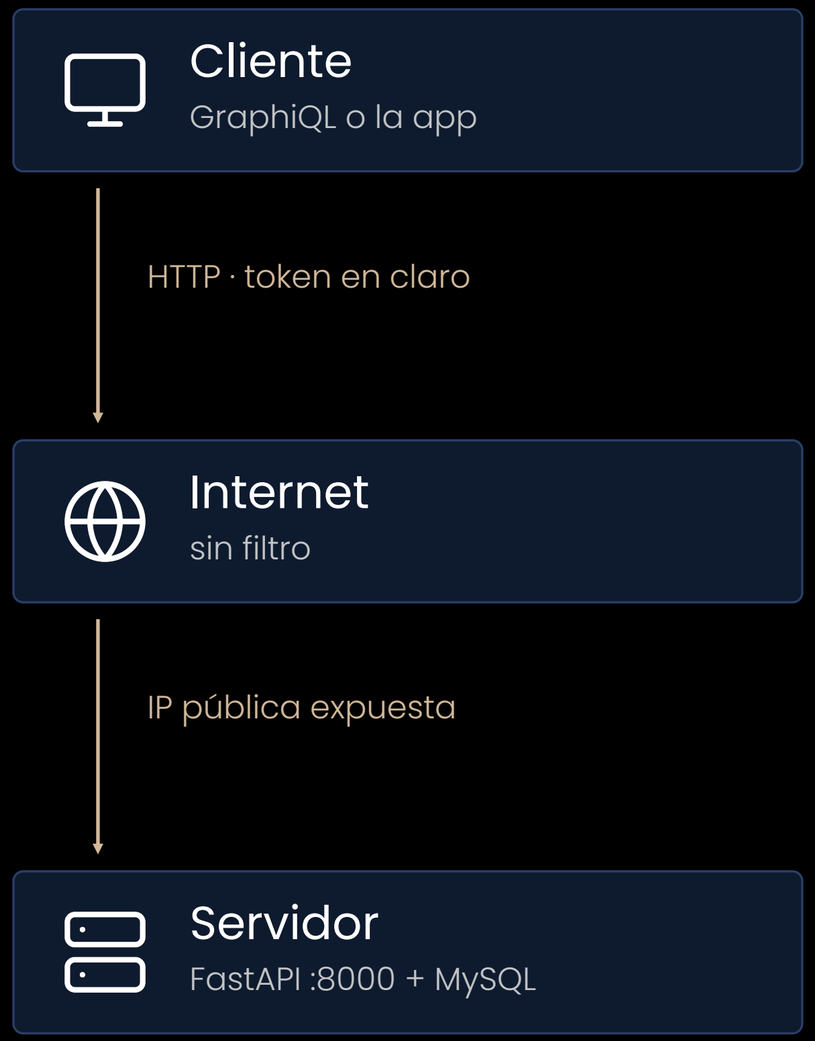
Piezas y pasos

03 Sergio Orion Huerta Martinez

Comparación y ejemplo

04 Diego Ramirez Magaña

Ganancias, límites y demo



EL PROBLEMA

Si publicamos la API tal cual, uvicorn queda abierto en el puerto 8000.

Sin HTTPS, el token JWT viaja en texto plano.

Con la IP, cualquiera puede saturar el servidor.

Cada petición repetida llega hasta FastAPI y MySQL.

QUÉ RESUELVE

HTTPS automático

Certificado gratis que se renueva solo. El token ya no viaja en claro.

Caché en el borde

Guarda respuestas GET cerca del usuario para que no lleguen a la API.

Protección del servidor

Oculta la IP real, frena ataques DDoS y filtra con reglas del WAF.

LAS PIEZAS

NUBE

Panel de Cloudflare: DNS, certificado, caché y reglas.

CONECTOR

cloudflared corre junto a la API y abre un túnel de salida.

CONFIGURACIÓN

docker-compose.yml con el token del túnel y sin puertos abiertos.

Cliente: GraphiQL o la app no cambian; solo usan https://ingresos.example.com

CÓMO SE USA

Pasos 1 y 2

Primero Cloudflare toma el DNS del dominio. Después el servidor se conecta a Cloudflare desde adentro: no abrimos ningún puerto.

1 · DOMINIO

Dar de alta el dominio en Cloudflare y cambiar sus nameservers.

2 · TÚNEL

Networking › Tunnels › Create Tunnel. Guardar el token.

3 · RUTA

Published application:
ingresos.example.com →
http://api:8000

4 · REGLAS

WAF administrado, límite de peticiones a /graphql y caché solo para lo público.

CÓMO SE USA

Pasos 3 y 4

Con eso la API responde en https:// y la IP del servidor deja de publicarse.

CON LO QUE YA VIMOS

Docker

Hoy el compose publica puertos. Con el túnel, cloudflared es un contenedor más y la API no expone ninguno.

Token y permisos

No los reemplaza. Cloudflare cuida el camino; leer_actor y exigir_permiso siguen decidiendo quién puede qué.

GraphQL

Todo va por POST /graphql y Cloudflare no guarda en caché un POST. La caché ayuda con GET y archivos.

MySQL

Nunca pasa por Cloudflare. Se queda en la red interna de Docker, sin salida a internet.

EJEMPLO

01

La API de ingresos

Agregamos cloudflared al docker-compose. La API deja de publicar el puerto 8000 y solo el túnel la alcanza.

```
services:
  db:
    image: mysql:8.4       # igual que hoy
  api:
    build: .
    command: uvicorn app.main:app --host 0.0.0.0
    # sin "ports": nadie entra directo
  tunnel:
    image: cloudflare/cloudflared
    command: tunnel --no-autoupdate run
    environment:
      TUNNEL_TOKEN: ${TUNNEL_TOKEN}

# Ruta en el panel de Cloudflare:
# ingresos.example.com → http://api:8000
```

EJEMPLO

02

El camino de una petición

La consulta accounts, con su token, pasa por cuatro paradas antes de leer la base.

GANANCIAS Y LÍMITES

$0

El plan gratuito ya incluye HTTPS, protección DDoS y Cloudflare Tunnel.

Lo que gana el equipo

HTTPS sin manejar certificados, la IP del servidor oculta y menos tráfico basura llegando a FastAPI.

Lo que cuesta

Hace falta un dominio propio, Cloudflare ve el tráfico descifrado y una regla de caché mal hecha puede mostrar datos de otro usuario.

CUÁNDO NO CONVIENE

01

Solo desarrollo local

Para probar en clase basta localhost; para enseñarlo, un túnel rápido temporal.

02

Datos que no deben pasar por terceros

Cloudflare termina el HTTPS: puede ver las peticiones descifradas.

03

Servicios internos

Dos backends en la misma red no necesitan un proxy público en medio.

DEMO

Túnel rápido a nuestra API, sin cuenta ni dominio

```
$ cloudflared tunnel --url http://localhost:8000
```

EQUIPO 4

FUENTES

Cloudflare Tunnel

https://developers.cloudflare.com/tunnel/get-started/

Default cache behavior

https://developers.cloudflare.com/cache/concepts/default-cache-behavior/

DNS · Proxy status

https://developers.cloudflare.com/dns/proxy-status/

¿PREGUNTAS?

Gracias · Equipo 4 · Cloudflare